

Basic (a simple version log) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real version log earns your trust). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A version log is just a record of the different variants of one prompt you've tried, each one with a name ("v1" , "v2"), the actual prompt text, and when it was saved. Save that as a small list of dictionaries in a JSON file — nothing fancier than that.

Things to use:

- A dictionary for each version: `{"version_name": "v1", "prompt_text": "...", "created_at": "..."} .`
- A list to hold more than one version.
- `json.dump(data, file)` to write it, `json.load(file)` to read it back.
- `datetime.now()` for a timestamp.

Intermediate Version

Two real Python skills carry this whole exercise: writing/reading a JSON file, and building up a list of dicts one entry at a time. `json.dump(data, f, indent=2)` writes readable JSON; `indent=2` is optional but makes the file human-readable if you ever open it directly. `json.load(f)` reads it back into the exact same Python structure (list of dicts) you saved.

For the timestamp, `datetime.now(timezone.utc).isoformat()` gives you a sortable, unambiguous string like `"2026-09-11T14:03:00+00:00"` — always store it as UTC, not local time, so it means the same thing no matter who reads the log later.

The exact pieces:

- `import json` — `json.dump(data, f, indent=2)` and `json.load(f)` .
- `from datetime import datetime, timezone` — `datetime.now(timezone.utc).isoformat()` .
- `pathlib.Path(path).exists()` — check whether the log file already exists before deciding whether to start a new list or load the existing one.
- A stand-in for Doc13's test suite: a small function, `run_eval_suite(prompt_text)` , that runs a handful of test cases against the prompt and returns a score. In this exercise you can stub it (fake, deterministic answers) so the script runs on its own — in real Project 5 code, this calls your actual Doc13 suite.

Advanced Version

Think about what a version log is actually *for*: the moment quality drops in production, you need to answer "which prompt was live when this happened" — and answer it with total confidence, not "probably v2, I think." That confidence only holds up if the log is **append-only**: once an entry is written, nothing ever edits or deletes it. If your save function can silently overwrite an old entry, the log can no longer be trusted, and the entire point of building it is lost.

The other real design question: what if the exact same prompt text gets saved twice under two different names by accident? A `hashlib.sha256(prompt_text.encode()).hexdigest()` stored alongside each entry makes that detectable — two entries with the same hash are the same prompt, whatever their names say.

The extra pieces needed for an append-only, hash-checked log:

- `hashlib.sha256(text.encode("utf-8")).hexdigest()` — a short fingerprint of the prompt text, cheap to compare.
- Load-modify-save instead of overwrite-blind: read the existing list first (or start a new one if the file doesn't exist yet), append the new entry, then write the whole list back — never open the file in a mode that truncates it before you've read what was already there.
- `sqlite3` as the more realistic alternative to a flat JSON file once more than one process might write to the log at the same time (`sqlite3.connect(path)`, `INSERT INTO versions (...) VALUES (...)`, then `conn.commit()`) — a database handles concurrent writes safely; a JSON file, read and rewritten whole, does not.

Sketch the append-only save function yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the plain idea and the tools for the tidy, single-writer case. Intermediate shows the real Python syntax for reading/writing that JSON file and building a trustworthy timestamp. Advanced adds what only matters once you take "this is the record that makes 'which version was live' answerable" seriously — append-only saving so history can't be quietly lost, a content hash to catch accidental duplicates, and SQLite as the realistic choice once more than one person or process writes to the log.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make an empty list called versions

make version v1: {"version_name": "v1", "prompt_text": "...", "created_at": now}
make version v2: {"version_name": "v2", "prompt_text": "...", "created_at": now}
add both to versions

save versions to versions.json with json.dump

for each version in versions:
    run the eval suite against its prompt_text
    print the version name and its score
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# version_log_practice.py
import json
from datetime import datetime

def run_eval_suite(prompt_text):
    # stand-in for Docl3's real test suite
    if "polite" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}

versions = []
versions.append({"version_name": "v1", "prompt_text": "Answer the customer's ques
versions.append({"version_name": "v2", "prompt_text": "Answer the customer's ques

with open("versions.json", "w") as f:
    json.dump(versions, f, indent=2)

for version in versions:
    result = run_eval_suite(version["prompt_text"])
    print(version["version_name"], result["score"])
```

Expected output:

```
v1 0.6  
v2 1.0
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
function save_version(name, prompt_text, path) -> None:
    load existing list from path, or start a new one if the file doesn't exist
    append {"version_name": name, "prompt_text": prompt_text, "created_at": utc n
    write the whole list back to path

function load_versions(path) -> list[dict]:
    if file doesn't exist: return empty list
    read and return the JSON list

save_version("v1", ..., "versions.json")
save_version("v2", ..., "versions.json")

for version in load_versions("versions.json"):
    run the eval suite, print name and score
```

```
# version_log_practice.py
import json
from datetime import datetime, timezone
from pathlib import Path

def save_version(name: str, prompt_text: str, path: str) -> None:
    versions = load_versions(path)
    versions.append({
        "version_name": name,
        "prompt_text": prompt_text,
        "created_at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(versions, f, indent=2)

def load_versions(path: str) -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)
```

Now write the loop that calls `run_eval_suite` for each saved version and prints its score, and compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
function save_version(name, prompt_text, path) -> dict:
    versions = load_versions(path)
    prompt_hash = sha256(prompt_text)
    if any existing entry already has this exact prompt_hash:
        print a warning, but still save it (names can differ on purpose)
    build the new entry with version_name, prompt_text, prompt_hash, created_at
    append it, write the whole list back
    return the new entry

everything else same as Intermediate, plus:
    print a small table: version_name | score | passed/total | created_at
```

Here's almost the whole thing — fill in the missing dedupe warning yourself:

```
# version_log_practice.py
import hashlib
import json
from datetime import datetime, timezone
from pathlib import Path

def load_versions(path: str) -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def save_version(name: str, prompt_text: str, path: str) -> dict:
    versions = load_versions(path)
    prompt_hash = hashlib.sha256(prompt_text.encode("utf-8")).hexdigest()

    # your turn: loop over `versions`, and if any existing entry's
    # prompt_hash matches this one, print a warning that this prompt
    # text was already saved under a different name – don't block the
    # save, just warn

    entry = {
        "version_name": name,
        "prompt_text": prompt_text,
        "prompt_hash": prompt_hash,
```

```
        "created_at": datetime.now(timezone.utc).isoformat(),
    }
    versions.append(entry)
    with open(path, "w") as f:
        json.dump(versions, f, indent=2)
    return entry
```

Fill in the dedupe warning yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same underlying idea (build a dict, save it, run the suite against it) at 3 completeness levels — Basic overwrites `versions.json` fresh every run with just the 2 versions it built this time, Intermediate turns that into reusable `save_version / load_versions` functions that load-then-append so old entries survive across runs, and Advanced adds a content hash so an accidental duplicate prompt saved under a new name gets caught, not silently hidden.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)